



Harness 101：写给 Agent 的虚拟文件系统

1. 前言

如今主流的 Harness / General Agent，底座几乎都是 Coding Agent。Cursor、Claude Code、Devin 抽掉「会写代码」这个能力，剩下的躯壳就薄得只剩一个对话框。底下挂着的，是清一色的 Sandbox——一个能跑 bash、能 npm install、能起一个浏览器的全功能盒子。

但笔者最近在做几个**并不需要执行代码**的 General Agent 时，开始怀疑这套默认配置。一个写作助手，它需要的是读 Markdown、写 Markdown，外加翻一翻历史草稿；一个日程助手，它要的只是读日历、写日历；一个客服 Agent，干的不过是查工单库、回写状态。这些场景里，Sandbox 是个昂贵的副作用——**启动慢、资源占用大、攻击面广**，而 Agent 真正需要的，只是 I/O。

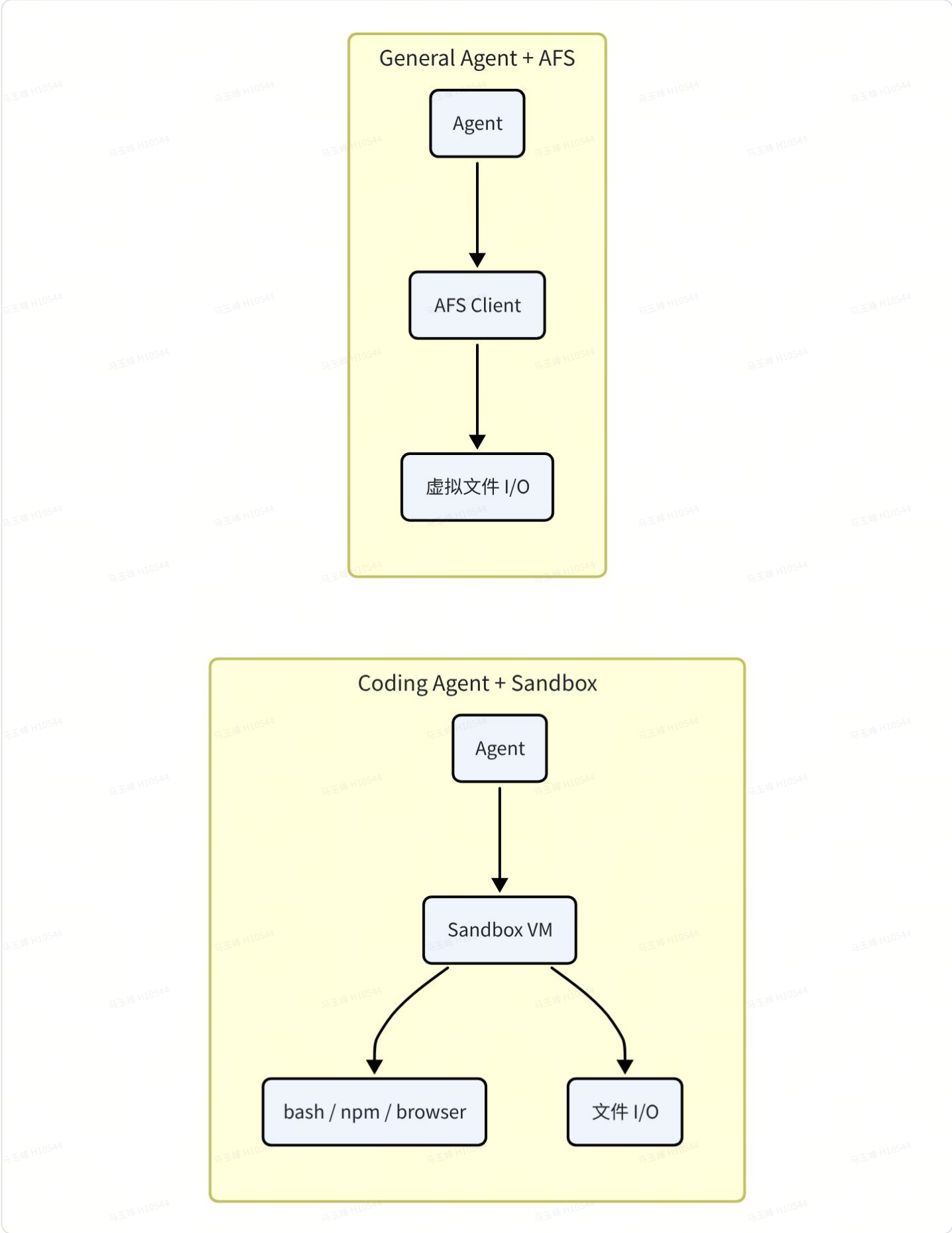
于是问题就摆在桌面上：Harness 一定要有基于 Container 或 MicroVM 技术的 Sandbox 吗？能不能把 Sandbox 拆成两层？让需要执行代码的 Agent 用厚的那层，让只需要读写数据的 Agent 用薄的那层？

1.1 来自 Linux 的启发

答案其实在 Unix 老哲学里躺了四十年——「一切皆文件」。`/dev/null`、`/proc/self/status`、`/sys/class/net/eth0/address` 这些「看起来是文件，实际是接口」的东西，正是把异构 I/O 抹平成统一抽象的经典招式。[Plan 9](#) 把这套哲学推到极致，连网络连接都是 `/net/tcp/clone` 这样的文件。Agent 时代，我们不妨再来一次：把所有 Agent 需要触碰的世界——用户文档、企业知识库、日历、工单、Session 临时数据——都暴露成虚拟文件，让 Agent 用最朴素的 `read` / `write` / `ls` / `grep` 就能完成绝大多数工作。

本文提议一个面向 General Agent 的轻量底座：**AFS (Agent File System)**。它不替代 Sandbox，而是从 Sandbox 里抽出 I/O 这层，让不需要执行能力的 Agent 不必再为一个完整的盒子买单。

下面这张图大概说明了这两条路的差别：



笔者的立场是：**AFS 是 Agent I/O 的最小公分母，不是所有 Agent 都需要 bash**。本文按背景、设计、协议、结语四章展开。

 你知道吗—— `/proc/self/status` ?

这是 Linux 上「一切皆文件」最经典的一张名片。`cat /proc/self/status` 会吐出当前进程的实时状态：PID、内存占用、线程数、信号掩码……每一行都是内核数据结构的字段，但你只用了一个 `cat`。背后没有 syscall、没有 SDK、没有 schema 协商，只有一个看起来像文本的虚拟文件。这套抽象的妙处是：**任何能 `read` 的程序，都自动成了运行时监控工具。**

加入 Agentara 2 群:



Agentara Underground 情报站 (2群) 外部

每日投喂AI/Agent新闻，不定期推送热点深度研...

群名片

加入 Agentara Underground 情报站

加群获取更多线上资料，和 7,500+ 位 AI 爱好者一起学习 Prompt、Context Engineering、Harness Engineering 和 Skills。

已经加入过 1 群的小伙伴不用重复加入，也请分享给需要的同学们

 同步块 [该内容不支持导出查看]

2. 背景

先看为什么默认装一个 Sandbox 不划算，再看 Unix 当年是怎么用一套抽象解决类似问题的。

2.1 Sandbox 的成本三件套

把 Sandbox 当作 Agent 的默认底座，代价是实打实的。

启动延迟。哪怕是最轻的容器化 Sandbox，冷启动也要数百毫秒到数秒；如果走 microVM（Firecracker、gVisor 这一档），首次启动到能跑命令的 P50 普遍在秒级。对于一次会话只需要读两个文件、写一段总结的写作助手，这点开销不亚于「为了切一片面包先拉来一台冰箱」。

资源占用。Sandbox 要预留 CPU、内存、磁盘镜像、网络命名空间。一个空闲实例就要吃掉百兆级内存，几千个并发会话乘起来，账单很难看。更尴尬的是大多数时间这些资源都在闲置——Agent 在等

LLM 出 Token，Sandbox 在等 Agent 调命令。

攻击面。一旦给 Agent 一个能跑任意命令的盒子，安全边界就从「数据访问控制」滑向「容器逃逸防御」。前者是数据库 ACL 的事，后者是内核 CVE 的事，复杂度天差地别。对一个只需要读企业 Wiki 的客服 Agent，背这套安全债毫无意义。

2.2 不一定需要 Sandbox 的 Agent，比想象中多

前言里的三类 Agent（写作、日程、客服）并不是孤例。任何一个工作模式可以归纳为「读资源、改资源、写回去」的业务 Agent——销售、HR、法务、运营——都吃不消 Sandbox 的全套设备。它们要的不是「执行能力」，而是「**对世界的统一可寻址 I/O**」。给它一个 Sandbox，等于给一个只想查字典的人塞了一台带显卡的工作站。

2.3 Unix 给过一次答案

「一切皆文件」是 Unix 留给后人的最大遗产之一。stdin / stdout / stderr 是文件，socket 是文件，块设备是文件，`/proc` 是文件，`/sys` 是文件——这套抽象的精髓在于：**任何 I/O 都能用 `read` / `write` 表达**，于是 `cat`、`grep`、`tee` 这些通用工具就自动适配了一切新出现的资源类型。

下面是几个经典的「伪文件」，它们看起来是普通文件，其实是内核挂出来的接口：

代码块

```
1 /dev/random          # 读它就拿一段熵；写它给内核回填熵池
2 /dev/null             # 写它丢掉；读它得 EOF
3 /proc/cpuinfo         # 读它拿当前 CPU 的型号、核心数、flags
4 /proc/self/maps       # 读它拿当前进程的虚拟内存映射
5 /sys/class/net/eth0/address # 读它拿网卡 MAC 地址
6 /sys/class/power_supply/BAT0/capacity # 读它拿电池剩余百分比
```

这些路径背后没有真实文件，只有一个内核回调——但因为它们长得像文件，你就能用 `cat`、`watch`、`grep`、`find` 来观察整个系统，而不需要再为「看一眼电池电量」单独写一个 C 程序。

Plan 9 把这套哲学推到了极致：**网络连接是文件**（`/net/tcp/clone`），**远程进程是文件**（通过 9P 协议挂载到本地命名空间），**窗口是文件**（写入即绘制）。一个 shell 脚本就能起 TCP 连接、做 RPC、操作远程机器，因为所有资源都已经被压扁成同一种 read / write 接口。详见 [The Use of Name Spaces in Plan 9](#) 与 [Plan 9 from Bell Labs - Wikipedia](#)。

你知道吗——`/proc` 与 Plan 9?

`/proc` 文件系统最早出现在 UNIX 8th Edition（Plan 9 的前身），由 Tom J. Killian 提出。核心创新是把进程和内核状态暴露成虚拟文件，而不是新增一堆 syscall——这意味着 `cat`、`grep`、`awk` 这些老工具瞬间就能查看运行时状态，省下无数 C 程序。Plan 9 后来把这一招用到了网络、显示、远程计算的方方面面，达到了「整台机器（甚至整个机房）」

都是一棵文件树」的境界。今天的 Linux 容器、Docker、Kubernetes Pod，本质上都还在吃这棵树的红利。详见 [proc\(5\) - Linux manual page](#)。

Agent 时代，我们手里的「资源」从设备和进程，变成了知识库、用户文档、日历、工单、Session 缓存。技术细节换了，**问题的形状没变**：异构 I/O 太多，需要一层统一抽象。AFS 想做的，就是把这层抽象重新搭一次——这次的客户不是 shell，而是 Agent。

3. 设计

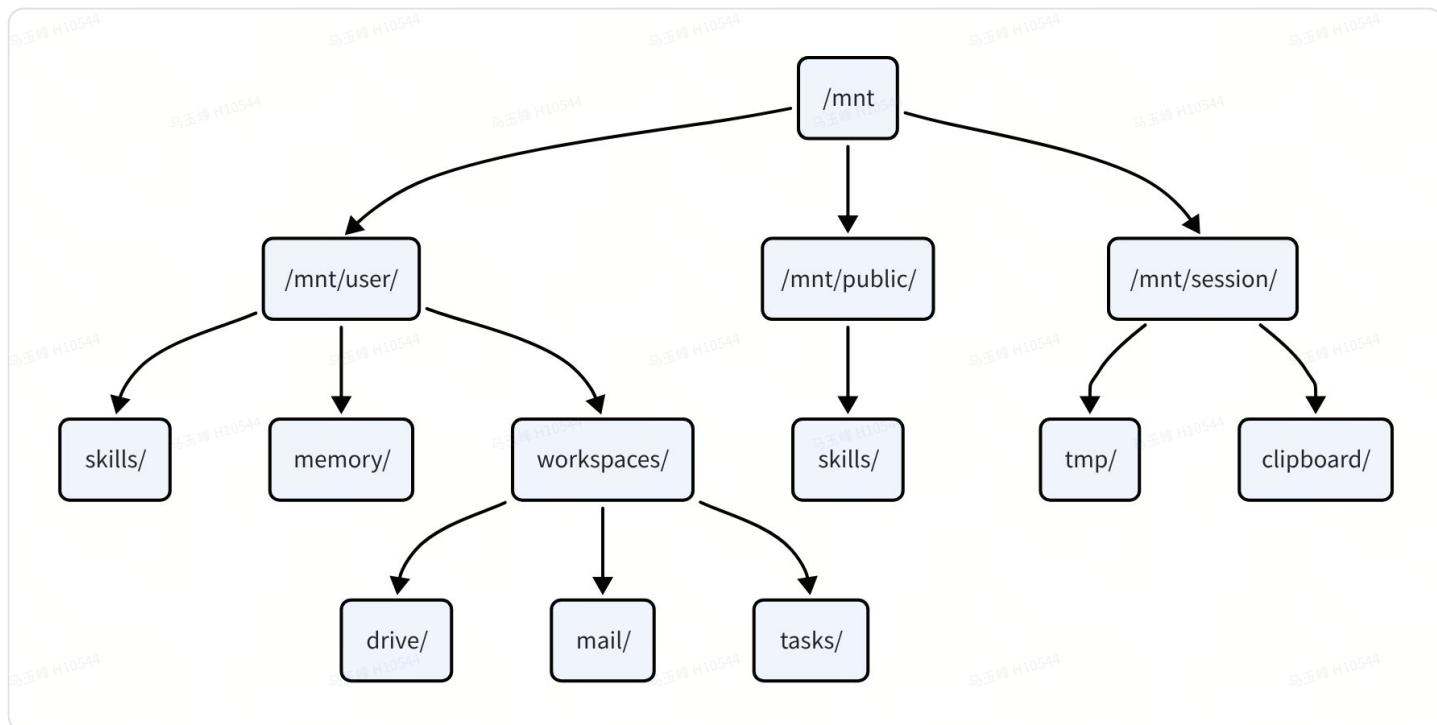
既然客户从 shell 换成了 Agent，三件事必须一次定清楚：**路径长什么样**、**Service 怎么派发**、**Provider 要满足什么协议**。把这三件事确定下来，AFS 就有了骨架；剩下的云盘、邮件、日历都只是往骨架上挂肉。

3.1 路径命名空间

AFS 内置三大 namespace，全部以 `/mnt/` 开头，借的是 Linux mount point 的直觉。

- `/mnt/user/`：以 `user_id` 隔离的长期空间。子树下规划好三个保留目录——`/mnt/user/skills/`（用户私有的 Skill 包）、`/mnt/user/memory/`（长期记忆 / 偏好 / SOUL.md）、`/mnt/user/workspaces/`（用户的工作目录，可以再 mount 云盘、邮箱等外部 Provider）。Agent 端拿不到原始 `user_id`，只能通过路径访问，`user_id` 由 AgentFSService（下文简称 Service）在派发时自动注入。
- `/mnt/public/`：无需 `user_id` 也无需 `session_id` 的全局只读空间，典型用途是 `/mnt/public/skills/`——平台预置的、所有租户共享的 Skill 库。Provider 不需要做权限判断，**「公开」**这件事由路径前缀直接表达。
- `/mnt/session/`：session 级的临时空间，会话结束即销毁。子树下规划好两个保留目录——`/mnt/session/tmp/`（临时下载、计算中间产物）与 `/mnt/session/clipboard/`（Tool 之间共享的剪贴板）。`session_id` 同样由 Service 注入，Provider 看不到也不应该自己存。

我参考的是 **Claude AI（也就是 Claude 网站）**，我们在开源 Harness 项目 [DeerFlow](#) 中也是类似的实现，三条命名空间放在一起的样子是这样的。



这套命名空间的好处是把「谁能看到这个文件」写进了路径前缀。Agent 的 System Prompt 里只要约定一句「`/mnt/user/` 是你这个用户的私有空间」，模型就能自然推理出哪些路径该读哪些不该写，省掉一整套 ACL 解释成本。

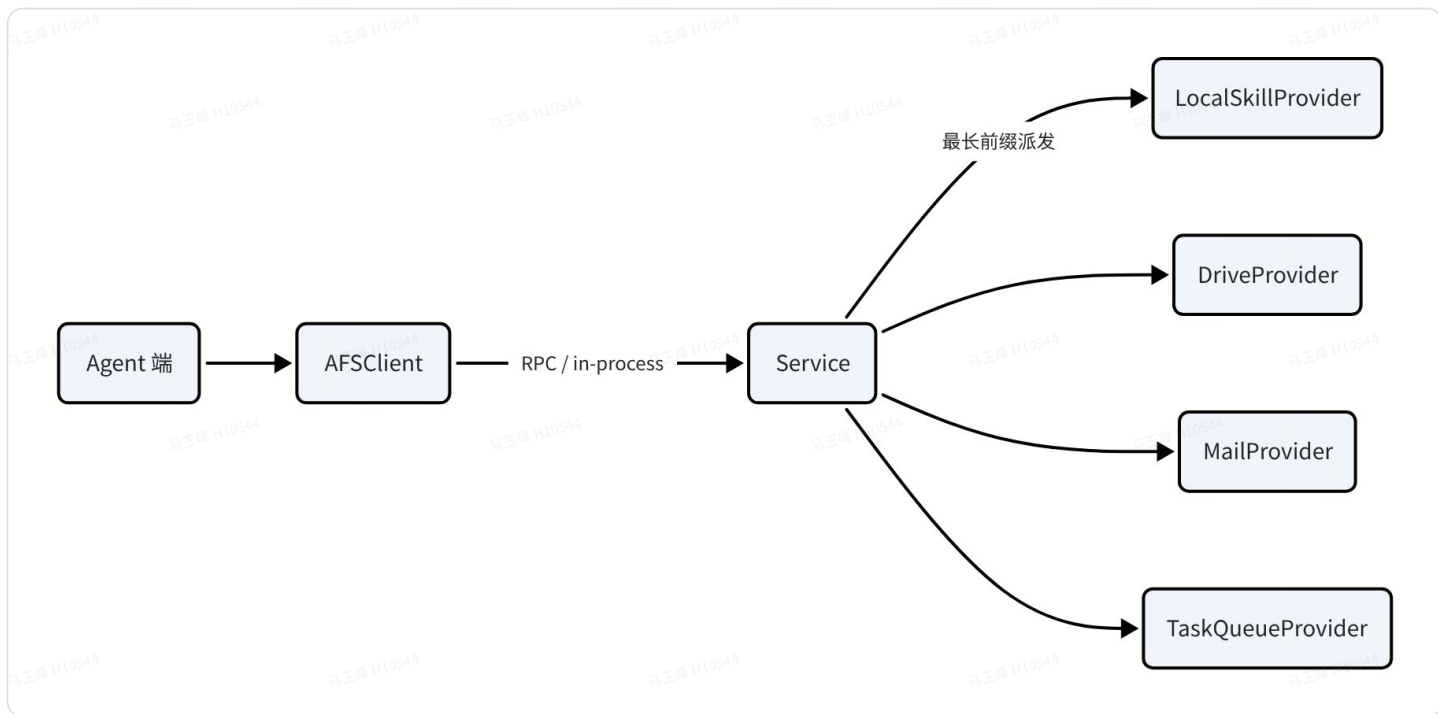
3.2 核心抽象 AgentFSService

骨架的中枢是 **AgentFSService**。它对外提供一个 `mount(absolute_path_pattern, provider)` 接口，把任意后端注册为某个路径子树的实现：

- `mount("/mnt/user/skills/", LocalSkillProvider(...))`
- `mount("/mnt/user/workspaces/drive/", DriveProvider(...))`
- `mount("/mnt/user/workspaces/mail/", MailProvider(...))`
- `mount("/mnt/user/workspaces/tasks/", TaskQueueProvider(...))`
- `mount("/mnt/session/", SessionTmpProvider(...))`

Agent 端发来的每一次 `read("/mnt/user/workspaces/drive/2026/q1/report.md")`，Service 都做一次**最长前缀匹配**，把请求派发到注册到 `/mnt/user/workspaces/drive/` 的那个 Provider，并把剩余路径 `2026/q1/report.md` 作为相对路径传下去。匹配规则与 Linux 内核的 `vfsmount` 树一致——后挂载的 mount point 覆盖前面的，子目录优先于父目录。

整个调用链画出来很短：

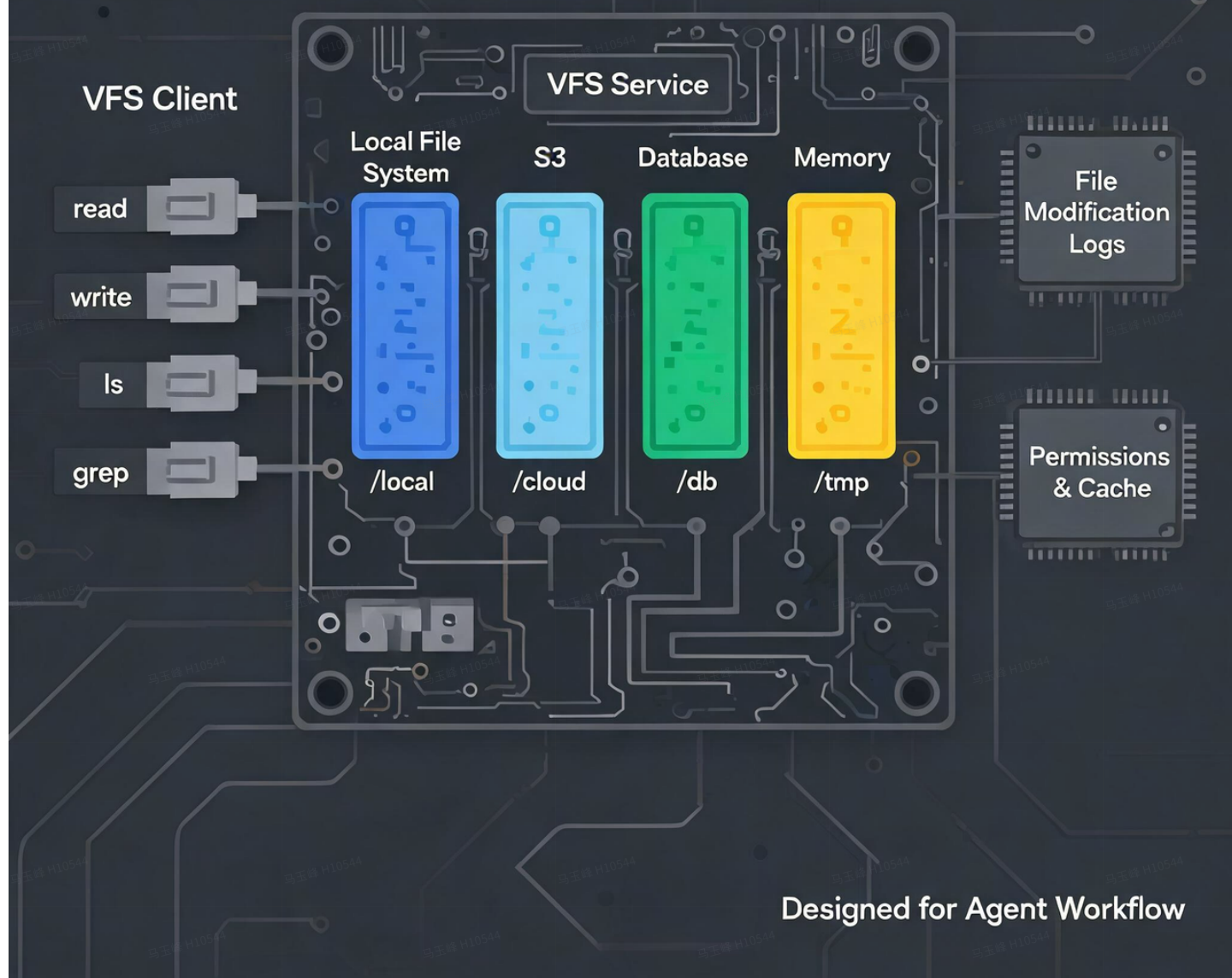


四个 Provider 在图里是**平行**的——这是设计上故意的：Service 不知道 DriveProvider 比 LocalSkillProvider 「更复杂」，它只看路径前缀和能力协议。新增一个 CRMProvider 不用改 Service 一行代码，只要 mount 上去。

但 mount 不是把任何对象都接得上——Provider 必须满足一个最小协议。

3.3 Provider 协议：能力靠类型表达

Virtual File System Architecture



最容易踩坑的是 Provider 的**能力差异**。云盘可写、邮件大多只读、任务队列支持 append 不支持 edit、知识库可能支持 grep——如果让每个 Provider 自己写 `supports_write = True` 这种声明字段，Service 就会被「能力开关」喂出一堆 if-else。

AFS 的做法是把能力做成 **Python Protocol**:

- **必选**: `read`、`ls` ——任何 Provider 都得能读、能列
- **可选**: `write`、`edit`、`grep`、`glob` ——通过独立 Protocol 声明，Provider 想支持就实现，不想支持就不写

mount 时 Service 用 `isinstance(provider, WritableProvider)` 一查，就知道这个挂载点要不要给 Agent 暴露 `write` 工具。Agent 端如果对一个只读挂载点调用 `write`，根本不会发出

请求——AFSClient 在工具发现阶段就拒掉了。**「能不能写」**这件事不需要 runtime 试错，类型系统已经回答了。

具体的 Protocol 签名留到下一章细写，这里先记住一个判断标准：**任何能力都应该可以靠「实现 / 不实现某个方法」来表达，而不是靠开关字段或继承层次**。这是 AFS 借自 Go 的 `io.Reader` / `io.Writer` 的经验——窄接口、组合优于继承。

4. 协议

骨架画完，接下来要把 Agent 端、Provider 端两侧的 Python 接口、典型调用、错误语义说清楚。AFS 的协议刻意做得很窄——窄到一个老练工程师扫一眼就知道该怎么实现，也窄到模型在 prompt 里读完接口签名就能正确调用。下面先看 Agent 端，再看 Provider 端，最后把派发、隔离、错误三件配套机制串起来。

4.1 Agent 端：AFSClient

Agent 拿到的入口只有一个：`AFSClient(session_id=...)`。它暴露 6 个方法，参数**全部是绝对路径**，不存在「当前工作目录」这种状态——Agent 不是 shell，给它 `cd` 只会让它在 token 上浪费理智。

代码块

```
1 from typing import Iterator, Optional
2
3 class AFSClient:
4     """Agent 端唯一入口; session_id 由 Service 在派发时注入."""
5     def __init__(self, session_id: str) -> None: ...
6     def read(self, path: str, *, range: Optional[tuple[int, int]] = None) ->
    str: ...
7     def write(self, path: str, content: str) -> None: ...
8     def ls(self, path: str) -> list[str]: ...
9     def edit(self, path: str, old: str, new: str) -> None: ...
10    def grep(self, path: str, pattern: str) -> Iterator[str]: ...
11    def glob(self, path: str, pattern: str) -> list[str]: ...
```

4.2 Provider 端：用 Protocol 声明能力

Provider 侧用 `typing.Protocol` 把能力做成结构化类型，每个 Provider 实现自己支持的那部分，不要继承也不需要 `register`。

```

1  from typing import Protocol, runtime_checkable, Iterator
2
3  @runtime_checkable
4  class BaseProvider(Protocol):
5      """所有 Provider 必选实现的最小集。"""
6      def read(self, rel_path: str) -> str: ...
7      def ls(self, rel_path: str) -> list[str]: ...
8
9  @runtime_checkable
10 class WritableProvider(BaseProvider, Protocol):
11     """声明此 Provider 支持写入与原地编辑。"""
12     def write(self, rel_path: str, content: str) -> None: ...
13     def edit(self, rel_path: str, old: str, new: str) -> None: ...
14
15 @runtime_checkable
16 class SearchableProvider(BaseProvider, Protocol):
17     """声明此 Provider 支持内容检索与文件名通配。"""
18     def grep(self, rel_path: str, pattern: str) -> Iterator[str]: ...
19     def glob(self, rel_path: str, pattern: str) -> list[str]: ...
20
21 class AgentFSService:
22     def mount(self, absolute_path_pattern: str, provider: BaseProvider) ->
None: ...
23
24 # 概念示例：把日历挂到 /mnt/user/workspaces/calendar/
25 # service.mount("/mnt/user/workspaces/calendar/", CalendarProvider(...))

```

代码加起来三十行不到，整个协议就交代干净了。

你知道吗——FUSE?

FUSE (Filesystem in Userspace) 是 Linux 内核 2.6.14 引入的一套机制，把「用户态进程实现文件系统」做成了正式的内核接口。sshfs、s3fs、rclone mount 都是它的产物——你看到的是一个普通目录，背后跑的是 SSH、S3、Google Drive。AFS 在思路上是 Agent 时代的 FUSE，把 Provider 替成用户态 Python 实现、把 syscall 替成 Tool 调用，但避开了和内核打交道的那堆 ioctl 与权限模型，剩下的都是熟悉的味道。

下面把能力发现、Session 隔离、一次完整调用、错误语义四件事——讲透。

4.3 能力发现：mount 时一次 isinstance

Service 在 `mount` 时对 provider 做 `isinstance(provider, WritableProvider)` 与 `isinstance(provider, SearchableProvider)` 两次检查，结果记在挂载表里。**Agent 端的 Tool 列表是 Service 根据这张表动态生成的**——一个挂载点如果只是 `BaseProvider`，那

Agent 在它子树里就只有 `read / ls` 两个工具可调。这就是为什么设计章说 Agent 不会对只读挂载点发起 `write`：模型看到的工具描述里压根没有这一项。

4.4 Session 隔离：路径前缀决定注入哪个 ID

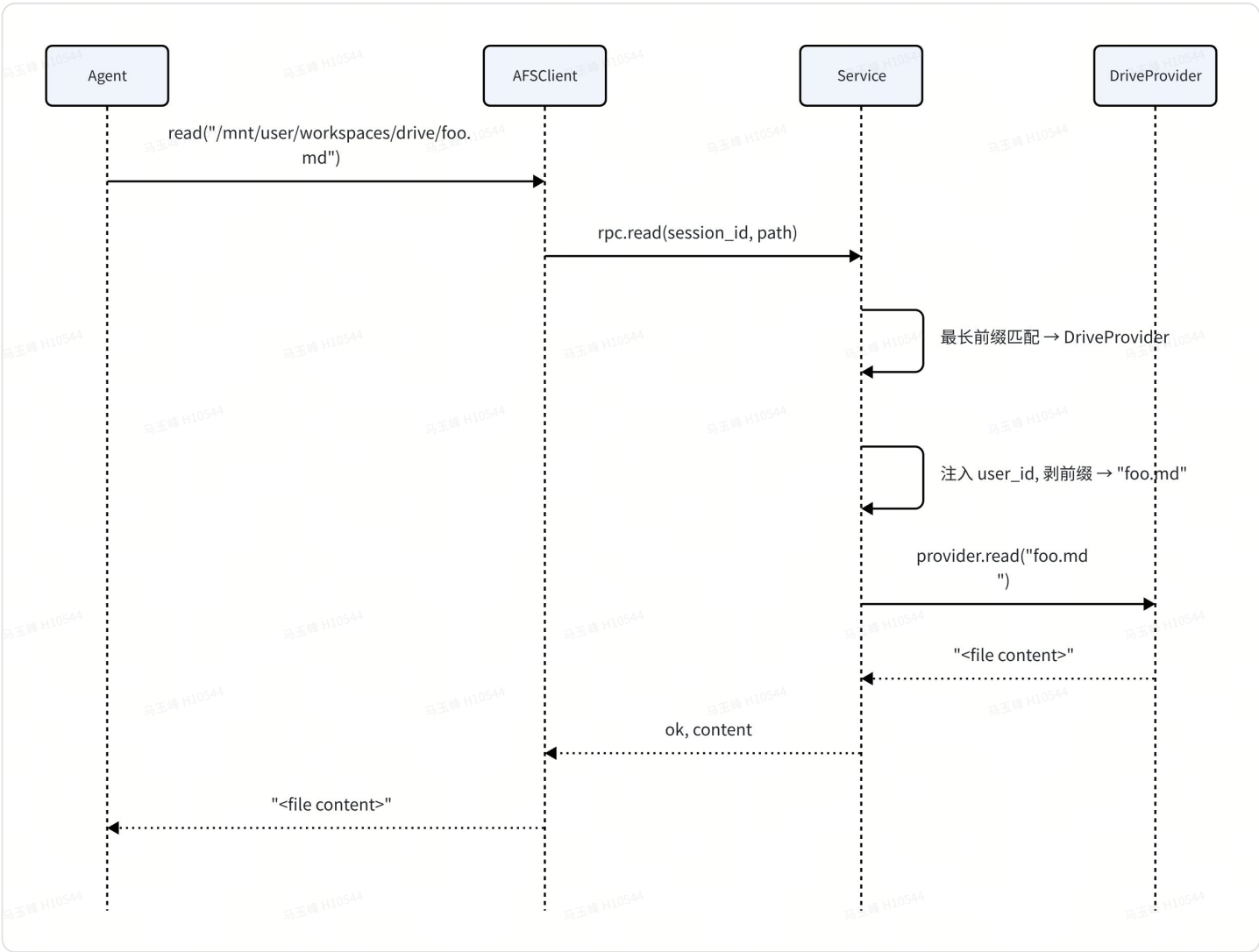
Service 派发请求时按路径前缀决定注入什么上下文：

- `/mnt/user/...` → 从 AFSClient 的 session 反查 user_id 注入
- `/mnt/session/...` → 注入 session_id
- `/mnt/public/...` → 不注入任何身份

Provider 永远拿到的是去掉前缀后的相对路径加上身份上下文，**自己不需要解析 user_id / session_id**，也无法越权访问别的用户。这层是 AFS 比裸 FUSE 多出来的部分，本质是把多租户语义内化到 mount 表里。

4.5 一次 read 的派发流程

把 Agent 调一次 `client.read("/mnt/user/workspaces/drive/foo.md")` 的时序串起来看，会更直观：



整条链路里，**Provider 始终活在自己的相对路径世界里**，看不到 user_id 之外的身份、看不到挂载点是什么、也不知道是哪条 session 在调它。这种「狭窄视野」就是良好抽象的标志：Provider 实现者关注的事和 Agent 关注的事完全不重叠。

4.6 错误语义：Provider 抛 Python 原生异常

Provider 端只抛 **Python 原生异常**：`FileNotFoundError`、`PermissionError`、`IsADirectoryError`、`NotImplementedError`。Service 拦下来翻译成统一的 tool result:

Python 异常	tool result error code
<code>FileNotFoundError</code>	<code>not_found</code>
<code>PermissionError</code>	<code>forbidden</code>
<code>IsADirectoryError</code>	<code>is_a_directory</code>
<code>NotImplementedError</code>	<code>unsupported_operation</code>
其它未捕获	<code>internal_error</code>

这样做有两个好处：Provider 作者写起来像写普通 Python，没有任何 AFS-specific 的错误类要 import；Agent 端看到的错误码统一、可枚举，方便在 System Prompt 里教模型「碰到 `forbidden` 就别再试同一条路径了」。

你知道吗——能力发现 (capability discovery) ?

POSIX 通过 `statvfs(3)` 暴露挂载点的能力，标志位里藏着 `ST_RDONLY`（只读）、`ST_NOEXEC`（禁止执行）、`ST_NOSUID` 等等——内核不让每个文件系统自己声明「我支不支持写」，而是把这件事做进 mount 表，调用方一次 syscall 就能问出来。AFS 借的是同一思路：能力探测属于挂载点这一层，不属于 Provider 自己。Provider 只负责实现方法，「这个挂载点能不能写」由 Service 用 `isinstance` 当场判定。

至此，Agent 端、Provider 端、Service 三方的接口、能力、错误语义都已落地。

加入 Agentara 2 群:

加入 Agentara Underground 情报站

加群获取更多线上资料，和 7,500+ 位 AI 爱好者一起学习 Prompt、Context Engineering、Harness Engineering 和 Skills。



Agentara Underground 情报站 (2群)

每日投喂AI/Agent新闻，不定期推送热点深度研...

群名片

已经加入过 1 群的小伙伴不用重复加入，也请分享给需要的同学们

同步块 [该内容不支持导出查看]

5. 进阶组合

立场写到这里有人会问：那需要执行能力的 Agent 怎么办？AFS 并不排斥 Sandbox，它只是把「执行」做成了可选的、分层的能力——按场景增配，而不是默认装满。这一章看 AFS 与不同重量 Sandbox 的接缝。

5.1 重栈：NAS / Git 后端打通 Container Sandbox

如果某个 Provider 背后是 NAS、企业 Git 仓库、Lustre 这类传统文件系统，`read / write / ls` 已经能解决八成日常，但剩下两成需要更复杂的动作——在仓库里跑 `pytest`、用 `git rebase` 整理分支、用 `rsync` 同步一份大目录。这种时候最干净的做法是把 AFS 的 mount 树挂进

Container Sandbox：

- 容器里跑一个轻 FUSE 适配器，把 `/mnt/user/workspaces/repo/` 暴露成 sandbox 内部的 `/work`
- bash 看到的是熟悉的 POSIX 路径，模型照常 `cd /work && pytest`
- 所有读写最终经 FUSE → AFSCClient → Service → Provider，权限、`user_id`、`session_id` 由 AFS 在出口处把守

这种组合的好处是，Sandbox 不再需要自己实现 Git 拉取、NAS 挂载这些后端逻辑——它继承的是 AFS 已有的 Provider 生态。Agent 视角看到的是「我有个目录可以 cd 进去」，背后则是一整套多租户、Provider 化的 I/O 框架。

5.2 轻栈：Node.js / Python 配更小的沙箱

另一类场景比 NAS 简单得多。Agent 只是想：把一个 JSON 算一下、把 Markdown 用 `pandoc` 转一下、跑一段 SQL、调一个用户写的 Python 脚本。这种「短任务、纯计算」的执行能力不该用完整的 Linux 容器，反而该用更轻的沙箱：

- **v8 isolate**: Cloudflare Workers 用的那一档，启动毫秒级、内存几 MB，跑 JavaScript / TypeScript
- **Pyodide / WASM Python**: 纯 WASM 沙箱，没有真实文件系统、没有真实网络，跑离线 Python
- **Firecracker microVM**: 比 Docker 轻一档的 VM 风格隔离，仍然有完整 syscall 表

把这些轻沙箱加进栈里，Agent 拿到的执行工具长成这样：`run_js(code)` / `run_python(code)` / `query_sql(sql)`，参数是代码片段，「文件」仍然走 AFS。沙箱内部读写宿主资源时，全部翻译成 AFSCClient 调用——这意味着「沙箱里写的文件」和「Agent 直接 `client.write` 的文件」共享同一棵 mount 树，没有第二份状态。

5.3 三档怎么选

这三档不是互斥的，而是一棵决策树：

- 只读资源 / 简单读写 → 纯 AFS，不挂任何 sandbox
- 需要跑代码、且语言固定（Python、JS、SQL）→ 轻沙箱 + AFS，按语言选 isolate / WASM / microVM
- 需要 **bash**、需要任意工具链（git、make、ffmpeg）→ Container Sandbox + AFS，把 mount 树 FUSE 进容器

AFS 的位置始终在最底层——它是 I/O 抽象，不替代执行能力，但所有上层执行能力都透过它访问数据。「该不该上 sandbox」这件事，从一个底座选型问题，降级成了「按场景增配」的问题。

6. 结语

6.1 三条主线

把全文收拢一下：

- **Sandbox 不是 Agent 的最小公分母，VFS 才是**。需要执行任意代码的 Coding Agent 当然要重栈，但绝大多数业务 Agent——写作助手、日程助手、客服、销售——它们要的从来不是 `bash`，而是一组稳定的、能按路径寻址的读写能力。给这类 Agent 强行套 sandbox，是用大炮打蚊子，账单上每一分钱都在为没用上的能力买单。
- **Linux「一切皆文件」的哲学在 Agent 时代再次被验证**。`stdin` / `stdout` / `socket` / 设备 / `/proc` 都是文件，是因为「按路径寻址 + 顺序读写」是人类发明过最便宜的统一接口。今天我们让 Agent 读云盘、写邮件、查日历、提 Issue，最便宜的统一接口仍然是它。

- **AFS 真正的价值在 Provider 生态。**一旦 mount 协议稳定，云盘、日历、邮件、任务队列、CRM、知识库——任何一个能被「按路径列出 / 读 / 写」的系统都能变成 Agent 眼里的目录。Agent 的扩展不再靠堆 Tool，而是靠挂载新文件系统；新接入一个 SaaS，写一个 Provider 就够了，不用动 System Prompt、不用改 Agent 编排、不用上线一批新工具。

6.2 展望

下一步要解的不是新接口，是**时间维度**。今天的 AFS 只覆盖一次性的 read / write，但真实业务里 Agent 经常想要：

- **流式追加**——长任务的日志、LLM 的逐 token 输出，对应 Linux 的 `tail -f` 语义
- **事件订阅**——「收到新邮件就叫我」「这个文件被改了告诉我」，对应 `inotify(7)` 那一层

这两件事如果也能用「文件 + 路径」的语法表达——比如

`client.tail("/mnt/user/workspaces/mail/inbox/")` 阻塞返回新邮件流——AFS 就从静态目录长出了时间轴，Agent 第一次具备了**等待**的能力。这是后续值得展开的方向。

6.3 思考

抽象做得好不好，看一件事：**它能不能把不一样的东西按「用途相似」包成同一个壳**。文件系统是这种抽象的极简范本——磁盘、网络、内存、设备，没有一个长得像，但它们都有 read / write / open / close，于是 Unix 用半页 syscall 表统治了五十年。

Agent 时代我们最该重学的，不是 Transformer，而是 Unix。模型已经够聪明了；缺的是一套像样的、窄到每个工程师都愿意去实现的接口规范。AFS 只是这条路上的一小步——把「按路径读写」这件事，重新还给 Agent。